

message-brokers.ppt

Aula 5

Message Brokers, filas e eventos

Comunicação assíncrona, filas, tópicos e eventos no contexto de microsserviços.

01 / 29

Proximo

Objetivo da aula

Entender por que um serviço publica um evento em vez de chamar tudo por HTTP.

A pergunta principal: como avisar outros serviços sem deixar o cliente esperando tudo terminar?

Resultado final

Pedidos API

|

| publica pedido.criado

v

Message Broker

|

+--> Notificacoes

+--> Faturamento

+--> Analytics

HTTP funciona bem

Quando a aplicacao precisa de resposta imediata, HTTP continua sendo a escolha natural.

- Buscar um usuario.
- Criar um produto.
- Atualizar um cadastro.

O problema aparece no pedido

Criar o pedido e o ponto principal. O resto pode acontecer depois.

- Enviar e-mail.
- Baixar estoque.
- Iniciar faturamento.
- Atualizar metricas.

Acoplamento

A API de pedidos começa a conhecer servicos demais.

Se o e-mail cai, o pedido pode falhar mesmo estando valido.

Message broker

Uma aplicacao intermediaria que recebe mensagens e entrega para outros sistemas.

Pense nele como uma central de distribuicao: quem envia nao precisa saber todos os destinos.

Por baixo dos panos

- Desacopla quem envia de quem recebe.
- Permite comunicacao assincrona.
- Ajuda no roteamento e reprocessamento.

Producer

Quem envia a mensagem.

Pedidos API --> Message Broker

No exemplo da aula, o producer e a API de pedidos.

Consumer

Quem recebe e processa a mensagem.

```
Broker --> Notificacoes  
Broker --> Faturamento  
Broker --> Analytics
```

Cada consumer cuida da sua responsabilidade.

Evento

Uma mensagem dizendo que algo importante ja aconteceu.

```
pedido.criado
```

O nome costuma ficar no passado porque representa um fato do negocio.

Nao publique ordem

Evite

envie email

Prefira

pedido.criado

Fila

Mensagens aguardam processamento.

```
pedido.criado --> fila notificacoes --> consumer
```

Boa quando um trabalho precisa ser feito por uma instancia disponivel.

ACK

O consumer confirma: processei com sucesso.

Sem confirmacao, o broker nao sabe se deu certo.

RabbitMQ

Producer

|

v

Exchange

|

v

Queue

|

v

Consumer

RabbitMQ costuma encaixar bem em filas de trabalho e roteamento por regras.

Exemplo: plataforma de entregas

Problema

O pedido foi criado, mas enviar notificacao, confirmar loja e gerar cobranca nao precisam travar a tela.

Solucao

Usar uma exchange de pedidos e filas separadas para notificacao, faturamento e analytics. A API publica `pedido.criado` uma vez; cada consumer processa sua fila sem bloquear o cliente.

RabbitMQ ajuda quando existe trabalho claro para ser distribuido entre consumidores.

Exchange decide a fila

Routing key

Etiqueta da mensagem: `pedido.criado`.

Binding key

Regra da fila: quero `pedido.criado`.

Binding

A ligacao entre exchange, fila e regra.

```
orders.exchange -- pedido.criado --> notifications.queue
```

Direct exchange

Analogia

Etiqueta exata no pacote. Se a etiqueta bate, entra naquela fila.

Caso real

`pagamento.aprovado` vai direto para a fila de faturamento.

```
routing key: pagamento.aprovado
```

```
binding key: pagamento.aprovado
```

Fanout exchange

Analogia

Um aviso no alto-falante. Todo mundo conectado recebe.

Caso real

`pedido.criado` avisa notificacoes, analytics e antifraude.

```
pedido.criado
  +--> notifications.queue
  +--> analytics.queue
  +--> antifraud.queue
```

Topic exchange

Analogia

Filtro por assunto. A fila escolhe um padrao, nao uma etiqueta unica.

Caso real

Suporte recebe todo evento de pedido: `pedido.*`.

```
pedido.* recebe:  
pedido.criado  
pedido.cancelado  
pedido.entregue
```

Kafka

```
Producer --> Topic --> Consumer Group --> Consumer
```

O foco esta no fluxo de eventos ao longo do tempo.

Kafka costuma fazer sentido quando existe alto volume, historico e necessidade de replay.

Exemplo: plataforma de mobilidade

Problema

Muitas localizacoes, status de corrida e eventos chegando o tempo todo.

Solucao

Usar topicos para receber eventos como `corrida.iniciada`, `motorista.localizacao_atualizada` e `corrida.finalizada`. Diferentes consumer groups leem o mesmo historico para mapa em tempo real, cobranca e analytics.

Nao e sobre uma tarefa unica. E sobre acompanhar um fluxo continuo de eventos.

Topico no Kafka

```
topico pedidos
|
+--> grupo notificacoes
+--> grupo faturamento
+--> grupo analytics
```

O topico guarda eventos de um assunto. Varios grupos podem ler o mesmo historico.

Consumer group

Um grupo de consumers trabalhando junto no mesmo serviço.

Mesmo grupo

Divide o trabalho entre instancias.

Grupos diferentes

Cada grupo recebe sua leitura do evento.

Partition

Uma divisao interna do topico.

- Ajuda Kafka a processar mais eventos em paralelo.
- Kafka garante ordem dentro da partition.
- Para pedido, use `pedidoId` como chave.

RabbitMQ ou Kafka?

RabbitMQ

Fila, tarefa, roteamento simples.

Kafka

Topico, historico, replay, alto volume.

A escolha depende do comportamento esperado, nao apenas do nome da ferramenta.

Quando usar mensageria

- Uma acao dispara varias reacoes.
- Nem tudo precisa acontecer agora.
- Varios servicos reagem ao mesmo fato.

Evento, HTTP ou RPC?

HTTP

Comando ou consulta direta com resposta imediata.

Evento

Publica um fato e continua sem esperar os consumers.

RPC

Envia uma pergunta pela mensageria e espera uma resposta.

Se o sistema só precisa avisar que algo aconteceu, prefira evento. RPC mantém dependência temporal e deve ser usado quando a resposta realmente é necessária.

Cuidados de producao

Mensageria melhora o desacoplamento, mas exige pensar em repeticao e falhas.

- Idempotencia evita efeito duplicado.
- Retry precisa de limite.
- Dead letter tira erro do fluxo principal.
- Ordem deve ser garantida apenas onde o negocio realmente exige.

